Blender and Python

À. Vinacua
using some ideas from Carlos Andújar

Sunday 8th April, 2018

Applicable versions

This slides apply to versions of blender after 2.5x, using Python 3.y

Learning Python and Blender-Python

Python3

is an interpreted language widely used for scripting and for quick development. Complete applications, like some desktop widgets, for example, may be written in python.
An authoritative reference for the language is http://docs.python.org/3.5; go first to `http://docs.python.org/3.5/tutorial`.

Blender-python

Authoritative references for the API are `http://www.blender.org/documentation/blender_python_api_2_78_release/contents.html` and `http://www.blender.org/manual/extensions/index.html`.
`http://en.wikibooks.org/wiki/Blender_3D:_Noob_to_Pro/Advanced_Tutorials/Python_Scripting/Introduction` may come in handy as well.

First steps

interactive interpreter for simple experiments

```
alvar@elmsley: ~
alvar@elmsley:~$ python3
Python 3.2.3 (default, Oct 19 2012, 19:53:16)
[GCC 4.7.2] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> 3 * 7
21
>>> 7 / 3
2.3333333333333335
>>> x = ['a', 'b', 'something else', 1234]
>>> x
['a', 'b', 'something else', 1234]
>>>
```

Python basics

Python uses indentation to denote blocks

```
1     ...
2     if flag:
3         print('flag is set')
4     elif x < 0:
5         print("flag is not set, "
6               "but x is negative")
7     else:
8         print("all else failed!")
9     ...
```

Sets

with usual ops:

```
A = set(['apple', 'pear', 'orange'])
B = set(['apple', 'kiwi'])
A - B      # set difference
A | B      # set union
A & B      # set intersection
l = list(A|B)
l.sort()
```

Dictionaries

Dictionaries can use any immutable as keys:

```
direct = {'john':123, 'james':3.9, 'jane':'abc',
          3.14:'pi', (0,2,5):'a tripple'}
a = direct['jane']
direct[2.71828] = 'e'
del direct['james']
direct.keys()    # returns iterator over keys
'john' in direct # returns True
for k, v in direct.items():
    print(k, v)
```

Control structures

Expect the usual...

```
a, b = 0, 1
while b < 10:
    print(b)
    a, b = b, a+b

phrase = ['a', 'list', 'of', 'words']
for x in phrase:
    print('%s has %d characters' % (x, len(x)))

for a in range(10):
    print(math.sqrt(a))

for x in range(len(phrase)):
    print('%s has %d characters' %
        (phrase[x], len(phrase[x])))
```

Iterators

You can iterate on anything reasonable

```
for x in [1, 2, 3]:
    print(x)
for x in (1, 2, 3):
    print(x)
for x in "123":
    print(x)
for x in {'one':1, 'two':2, 'three':3}:
    print(x)
for x in open('examplefile.txt'):
    print(x) # x iterates over lines in the file
```

But beware the infinite loop!

wrong

```
1 for w in words:
2     if w[0:2]=='ab':
3         words.insert(0, w)
```

Use instead

```
1 for w in words[:]:
2     if w[0:2]=='ab':
3         words.insert(0, w)
```

which operates on a **copy** of words...

But beware the infinite loop!

wrong

```
1 for w in words:
2     if w[0:2]=='ab':
3         words.insert(0, w)
```

Use instead

```
1 for w in words[:]:
2     if w[0:2]=='ab':
3         words.insert(0, w)
```

which operates on a **copy** of words...

I/O

```
# input string (line)
ok = input('Do you agree? ')

# Output:
print("Something...")
print("Tell me:", end="")  # no endline
# formatted output:
print("%5.2f, some comment %03d" % (pi, index))
```

I/O (contd.)

```
# Files:
f  = open('name.txt')
output = open('file.out', 'w')
all = f.read  # whole file
ln = f.readline() # one line
for line in f:  # files are 'iterable'
    print(line)
f.close()
output.write('Something or other')
print('Something or other', file=output)
```

Functions

```
1 def square(n=1): #may include default
2     return n*n
3
4 n=3
5 square(5)
6 square(2)
7 square(n=6)   # may call by name
8 n    # the parameter is local
9 square() # uses default
```

parameter passing

Parameters are passed by reference. Direct assignment may thus not have the effect you expect.
Keep in mind that numbers and strings are immutable: you cannot modify them.

Visibility, scope. . .

may read but not write globals

```
ex1=5
def myfunc(n=1, str='default'):
    print(ex1)  # OK, uses global
    print(n, end=' ') #end w. ws instead of nl
    print(str)
    ex1 += 1     # ERROR (must remove to test)
myfunc()  # uses defaults
myfunc(str='other') # uses default 'n'
```

Modules

You can group a set of Python statements and definitions in a file `name.py` to create a module '`name`'.

contents of file `fibo.py`:

```
# Fibonacci numbers module

def fib(n): # return Fibonacci series up to n
    result = []
    a, b = 0, 1
    while b < n:
        result.append(b)
        a, b = b, a+b
    return result
```

It will be searched for in the path `sys.path`, which can be extended via the environment variable `PYTHONPATH`

Modules

You can group a set of Python statements and definitions in a file `name.py` to create a module '`name`'.

contents of file `fibo.py`:

```
# Fibonacci numbers module

def fib(n): # return Fibonacci series up to n
    result = []
    a, b = 0, 1
    while b < n:
        result.append(b)
        a, b = b, a+b
    return result
```

It will be searched for in the path `sys.path`, which can be extended via the environment variable `PYTHONPATH`

Modules (II)

You can then use these definitions elsewhere importing the module:

some other program, or interpret

```
import fibo
fibo.fib(100)
```

which would return

```
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

or also...

```
from fibo import fib
fib(100)
```

or even

```
from fibo import *    # not advisable...
fib(100)
```

Modules (II)

You can then use these definitions elsewhere importing the module:

some other program, or interpret

```
import fibo
fibo.fib(100)
```

which would return

```
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

or also...

```
from fibo import fib
fib(100)
```

or even

```
from fibo import *    # not advisable...
fib(100)
```

Classes

Class definition

```
class MyClass:
    """Documentation string..."""
    i = 1234  # attribute
    def __init__(self): # may have a constructor
        self.otherattr = {'one':1, 'two':2}
        self.i = 2
    def f(self):  # methods have extra param.
        return self.i*'hello '
```

Instantiation

```
x = MyClass()
x.f()
x.i = 3
x.__doc__
x.anythingYouMayWishToUse = [1, 2, 3]
```

Blender's Python API

It is an intricate, long, and often redundant API.

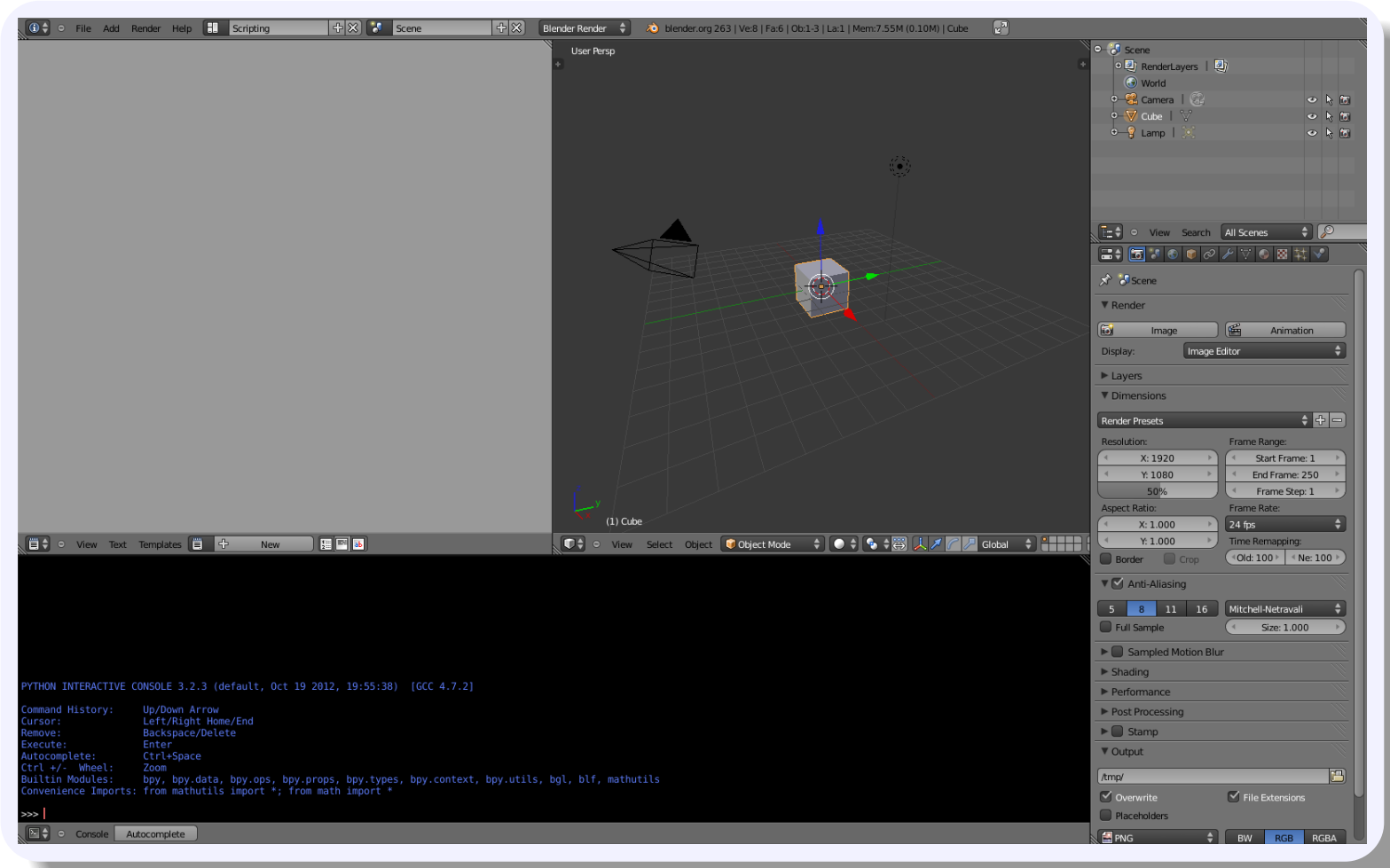
Main components

```
bpy.context -> object(s) selected, mode,...
bpy.data    -> the current file
bpy.utils   -> :-)
bpy.ops     -> all defined operators
bpy.types
...
mathutils   -> useful things like Vector, Matrix,
               Quaternion, ...
```

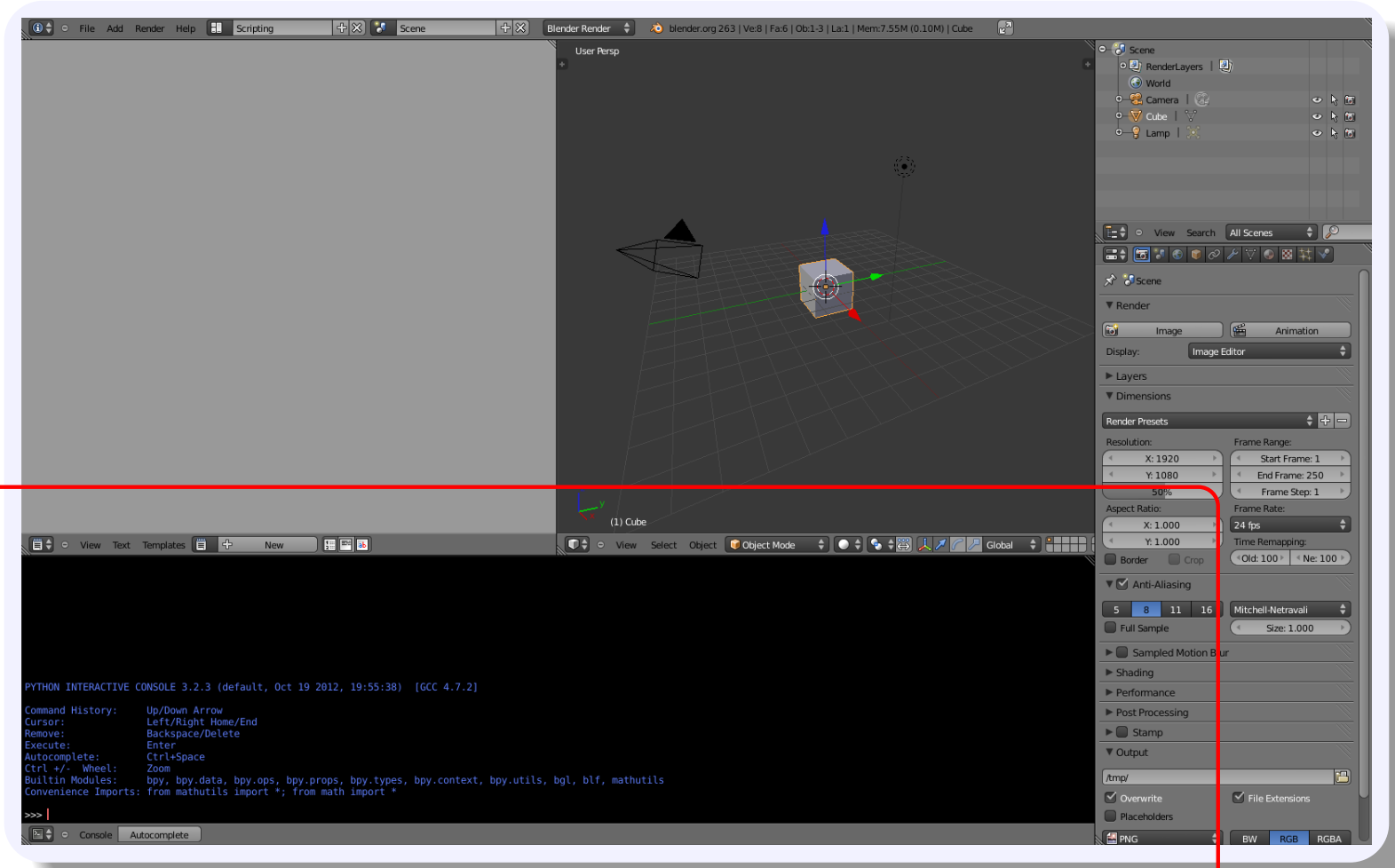
Some of this is still in flux.

Scripting layout

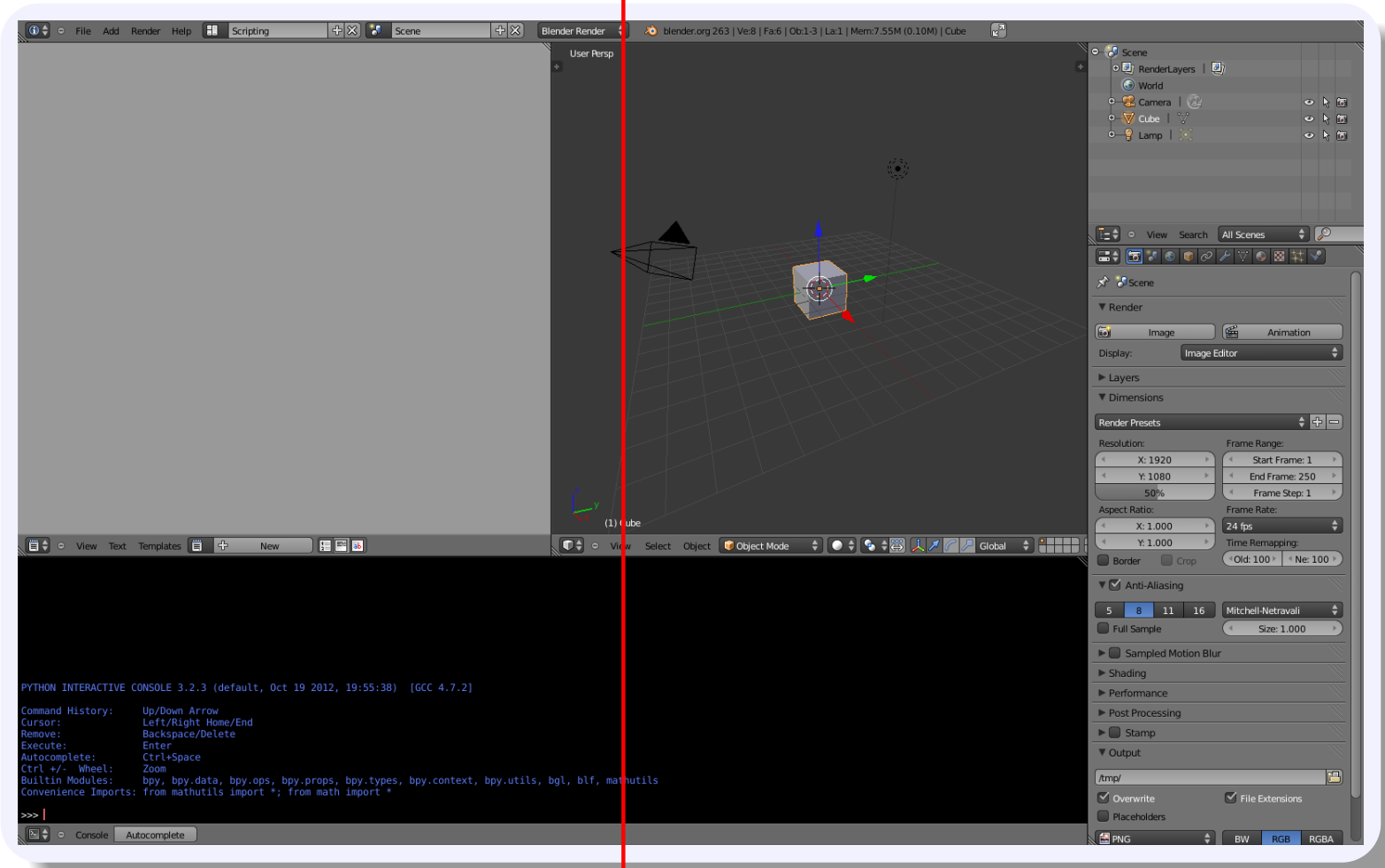
Click 'Ctrl-Right' to reach. . .



Scripting layout



Scripting layout



Experimentation

The black console at the lower left already has the main packages imported.
It allows interactive exploration and experimentation. Furthermore, it auto-completes pressing 'Ctrl-Space'.
Try entering

```
dir(bpy.types.Mesh)
help(bpy.types.Mesh.from_pydata)
type(bpy.data.objects['Cube'].data)
mesh = bpy.data.objects['Cube'].data
type(mesh.vertices[2])
mesh.vertices[3].co[2]+=5.
len(mesh.polygons)
```

Working with the editor

Beware that editing a script after having been run often crashes blender! There are, however, options:

Enter just this in blender's editor:

```
import imp
import sys
if not '' in sys.path:
    sys.path.append('')
import myscript
imp.reload(myscript)
myscript.main()
```

Or alternatively, use the script edit_external_for_blender to edit with emacs, and do not ever touch the editor's text. You can get this from `git://github.com/ldo/edit_external_for_blender.git` or from `/assig/sgi/Sessions/lab2/edit_external.py`

Meshes

Not to be confused with BMesh!

Mesh stores 4 main arrays:

- `Mesh.vertices` array of triples of floats
- `Mesh.edges` array of pairs of positive integers
- `Mesh.loops` array of corners of the mesh
- `Mesh.polygons` array of faces of the mesh

Each of them carries a lot of other information. For example, vertices store also `normals`. Notice that `loops` indexes corners, so its indices may be used elsewhere for things like texture coordinates. See `info_mesh.py` for use examples.